

Week 5 Assignment

Rahti Application Extension with Redis Cache

Three-tier Nginx + Flask/Gunicorn + MySQL application extended with Redis

Part A

1. **What is a Web Application Firewall (WAF) and what kinds of attacks does it defend against? Where would you place one in the three-container architecture you built in Week 4 (Route → frontend → backend → db)?**

A Web Application Firewall (WAF) is a security layer that examines HTTP and HTTPS requests before they reach a web application. It can detect and block common web attacks such as SQL injection, cross-site scripting (XSS), malicious request patterns and some forms of automated abuse. A WAF can use rules to inspect request paths, parameters, headers and request bodies and reject requests that match known attack patterns.

For the three-tier application built in Week 4, a WAF would be placed at the public edge, before the application frontend. Conceptually, the architecture would be Route/WAF → Nginx frontend → Flask backend → MySQL database. The WAF should protect the publicly reachable web entry point rather than being placed between the backend and MySQL. This would add another security layer while the backend and database remain internal to the OpenShift cluster.

2. **What is YAML and why has it become the default configuration format for Kubernetes/Rahti manifests? Compare it briefly to JSON.**

YAML is a human-readable data serialization format commonly used for configuration files. It represents structured information using indentation, keys and values. Kubernetes and OpenShift use YAML manifests to describe declarative resources such as Deployments, Services, ConfigMaps, Secrets and PersistentVolumeClaims. A manifest describes the desired state, and the platform works to make the running application match that state.

Compared with JSON, YAML is generally easier for people to read and edit because it uses indentation and does not require braces and quotation marks around every structural element. JSON is more strict and is widely used for machine-to-machine data exchange. For example, the Redis Deployment used in this assignment is described in a YAML manifest containing the Deployment name, Redis image, port and Redis server command. The same structure could be represented in JSON, but the YAML form is more convenient to maintain as a Kubernetes or Rahti configuration file.

3. **List some public APIs (see the example list of public APIs) and describe a realistic integration you could add to the backend you built in Week 4.**

Public APIs provide programmatic access to data or services offered by an external provider. Examples include weather APIs, public transport APIs, geographic services and open-data APIs. A realistic extension to the Week 4 Flask backend would be to call a public weather or open-data API and expose selected results through the application's /api endpoint.

For example, the Flask backend could request current weather information for a selected city, process the response and return a small JSON object to the frontend. In production, the application would need to consider API rate limits, timeouts, error handling and whether an API key is required. Any private API key should be stored in an OpenShift Secret rather than committed to Git. Caching could also reduce repeated external requests; this connects directly to the Redis component implemented in Part B.

4. What are open data portals (for example avoindata.fi or the European Data Portal) and why does choosing the right open-data format matter for reuse?

Open data portals publish datasets that can be accessed and reused by other people, organisations and applications. Examples include avoindata.fi and the European Data Portal. The data can be used for analysis, visualisation or as an input to software applications.

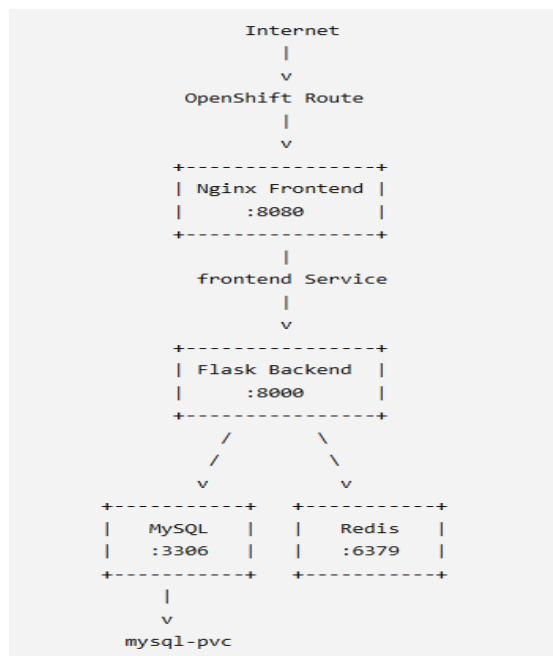
Choosing an appropriate open-data format matters because it affects how easily the data can be processed and reused. Machine-readable formats such as JSON or CSV are usually easier to use in software than a dataset that is only available as a formatted document. The structure and consistency of the data also matter: clear field names, stable identifiers and documented units make it easier for another application to consume the dataset correctly. For a Flask application, an API or machine-readable dataset could be retrieved by the backend, transformed if necessary and returned to the frontend.

Part B — Hands-On: Redis Cache Extension

1. Objective

The Week 4 application consisted of a Nginx frontend, a Flask/Gunicorn backend and a MySQL database running on CSC Rahti. For Week 5, Redis was added as a fourth orchestrated component. Redis is used as a cache by the Flask backend, while MySQL remains the persistent database.

2. Updated Architecture



The frontend is publicly accessible through the OpenShift Route. The Flask backend, Redis and MySQL communicate through internal OpenShift Services. Redis is not exposed publicly.

3. Redis Deployment and Service

Redis was deployed as its own OpenShift Deployment using the redis:7-alpine image. A separate ClusterIP Service named redis provides stable internal access to port 6379. The running Redis Pod was shown by `oc get pods`, and the Service and endpoint were verified with `oc get svc redis` and `oc get endpoints redis`.

The backend environment contains `REDIS_HOST=redis` and `REDIS_PORT=6379`. DNS resolution from the backend resolved the Redis Service to the cluster IP 172.30.152.1, demonstrating that the backend communicates with Redis through the OpenShift Service rather than directly using a Pod IP.

4. Redis Configuration Problem and Solution

Initially, Redis attempted to save RDB snapshots in `/data` and failed with a permission denied error. The Redis logs repeatedly reported that the temporary RDB file could not be opened. Because Redis was configured to stop writes when background saving failed, the Flask backend also received a `MISCONF` error when it attempted to write to Redis.

Redis was then configured specifically as a cache by disabling RDB snapshot saving and AOF persistence:

```
redis -server --save "" --appendonly no
```

After applying the updated Deployment, Redis started successfully and accepted normal cache operations. This configuration is appropriate for the cache role used here because MySQL and its `PersistentVolumeClaim` remain responsible for persistent application data. Redis should not be described as the persistent source of truth in this setup.

5. Backend-to redis Integration test

The Flask backend was tested directly against the Redis Service. The test returned `True` for `PING`, `True` for `SET`, and the stored value `'working'` for `GET`. This demonstrated that the backend could resolve the Redis Service, establish a connection and perform cache operations.

6. Cache Behaviour

The `/api` endpoint was tested twice from inside the backend Pod. The first request returned:

```
{"cache": "MISS", "message": "Hello from MySQL via Flask!", "mysql_server_time": "2026-09-26 20:13:16", "page_views": 14}
```

The same request was then made again. The second response returned:

```
{"cache": "HIT", "message": "Hello from MySQL via Flask!", "mysql_server_time": "2026-09-26 20:13:16", "page_views": 14}
```

The change from `MISS` to `HIT` demonstrates that the Flask backend is actually using Redis as a cache. The identical MySQL server time in the two responses is also consistent with the value being served from the cache on the second request.

7. Original Week 4 Application still Working

After the Redis integration, the public application was opened and refreshed successfully. The page displayed the MySQL server time 2026-09-29 10:26:04 and page views 21, together with the message 'Backend and database connection successful.' This verifies that the original frontend → backend → database flow remained functional after adding Redis.

A direct backend API test also returned HTTP 200 with a JSON response containing the message 'Hello from MySQL via Flask!', the MySQL server time and page_views. Therefore, the Redis extension did not prevent the original database functionality from operating.

8. Production Readiness Reflection

A key production concern with a Redis cache is cache invalidation. Cached data can become stale when the underlying MySQL data changes. The application therefore needs a suitable expiration time or an explicit invalidation strategy. The cache should improve response time and reduce repeated database work, but MySQL should remain the source of truth for persistent application data.

Reliability is another consideration. Redis is currently configured without persistent RDB or AOF storage because it is being used only as a cache. If a Redis Pod disappears, the application should be able to recover by reading the required data from MySQL and repopulating the cache. In production deployment, resource limits, monitoring, access control and appropriate availability/scaling policies would also need to be considered.

9. Evidence and Experiments

Evidence	Result
Redis Pod	Running as its own Deployment
Redis Service	ClusterIP Service on port 6379
Backend DNS resolution	redis resolves to the Redis Service
Backend → Redis PING	True
Redis SET/GET	Successful
First /api request	cache = MISS
Second /api request	cache = HIT
Public application	MySQL server time and page views displayed
MySQL functionality	HTTP 200 and database information returned

10. Conclusion

The Week 5 extension adds Redis as a separate orchestrated component rather than modifying the existing application container structure. Redis has its own Deployment and Service, and the Flask backend communicates with it through the internal Service name. The MISS/HIT experiment demonstrates actual cache use. At the same time, the original Week 4 application continues to retrieve MySQL data and display the server time and page views through the public application.

3-Tier Rahti Application

Frontend is running!

Database Information

MySQL server time: 2026-09-29 10:26:04

Page views: 21

Record Page View Refresh

Backend and database connection successful.

```
C:\Users\USER\oc>oc.exe get pods
NAME                                READY   STATUS    RESTARTS   AGE
backend-5877b6588-brmpn            1/1     Running   0           2d13h
frontend-d94bb77bd-cqjbf          1/1     Running   0           10d
mysql-5c44b4574c-57brh            1/1     Running   0           10d
rahti-webapp-3-build               0/1     Completed 0           20d
rahti-webapp-4-build               0/1     Completed 0           16d
rahti-webapp-5-build               0/1     Completed 0           16d
rahti-webapp-6-build               0/1     Completed 0           16d
rahti-webapp-679f469ff8-2lhsz     1/1     Running   0           16d
rahti-webapp-679f469ff8-gtk6b     1/1     Running   0           16d
rahti-webapp-679f469ff8-pwbng     1/1     Running   0           16d
rahti-webapp-7-build               0/1     Completed 0           16d
redis-6c9767f965-szrrb            1/1     Running   0           6m51s
```

```
C:\Users\USER\oc>oc.exe get svc
NAME      TYPE        CLUSTER-IP      EXTERNAL-IP  PORT(S)    AGE
backend   ClusterIP   172.30.48.204   <none>       8000/TCP   10d
frontend  ClusterIP   172.30.30.105   <none>       8080/TCP   10d
mysql     ClusterIP   172.30.156.110 <none>       3306/TCP   10d
rahti-webapp ClusterIP   172.30.92.62   <none>       8080/TCP   23d
redis     ClusterIP   172.30.152.1    <none>       6379/TCP   2d14h
```

```
C:\Users\USER\oc>oc.exe exec deployment/redis -- redis-cli SET test hello
OK
```

```
C:\Users\USER\oc>oc.exe exec deployment/redis -- redis-cli GET test
hello
```

```

C:\Users\USER\oc>oc.exe get pods
NAME                                READY   STATUS    RESTARTS   AGE
backend-5877b6588-brmpn            1/1     Running   0           30s
frontend-d94bb77bd-cqjbf           1/1     Running   0           8d
mysql-5c44b4574c-57brh             1/1     Running   0           8d
rahti-webapp-3-build                0/1     Completed 0           18d
rahti-webapp-4-build                0/1     Completed 0           14d
rahti-webapp-5-build                0/1     Completed 0           13d
rahti-webapp-6-build                0/1     Completed 0           13d
rahti-webapp-679f469ff8-2lhsz       1/1     Running   0           13d
rahti-webapp-679f469ff8-gtk6b       1/1     Running   0           13d
rahti-webapp-679f469ff8-pwbng       1/1     Running   0           13d
rahti-webapp-7-build                0/1     Completed 0           13d
redis-c46d5dffc-khg7f              1/1     Running   0           20m

C:\Users\USER\oc>curl.exe http://frontend-mywebapp-upeksha.2.rahtiapp.fi/api
{"cache":"MISS","message":"Hello from MySQL via Flask!","mysql_server_time":"2026-09-26 20:13:16","page_views":14}

C:\Users\USER\oc>curl.exe http://frontend-mywebapp-upeksha.2.rahtiapp.fi/api
{"cache":"HIT","message":"Hello from MySQL via Flask!","mysql_server_time":"2026-09-26 20:13:16","page_views":14}

C:\Users\USER\oc>curl.exe -X POST http://frontend-mywebapp-upeksha.2.rahtiapp.fi/api/view
{"cache":"INVALIDATED","page_views":15}

C:\Users\USER\oc>curl.exe http://frontend-mywebapp-upeksha.2.rahtiapp.fi/api
{"cache":"MISS","message":"Hello from MySQL via Flask!","mysql_server_time":"2026-09-26 20:14:00","page_views":15}

```

Public Application URL: <http://frontend-mywebapp-upeksha.2.rahtiapp.fi/>
 (Week 4 assignment content exists)

References:

1. <https://kubernetes.io/docs/>
2. <https://redis.io/>
3. https://en.wikipedia.org/wiki/Web_application_firewall
4. <https://www.imperva.com/products/web-application-firewall-waf/>